

Reviewer Pack — Minimal Number of Generators Invariant for Tseitin Resolutio...

Entry 2e69f7d2fd43 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 16:11:15 UTC

Minimal Number of Generators Invariant for Tseitin Resolution Lower Bounds

Entry ID: 2e69f7d2fd43 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 16:11:15 UTC

1. Conjecture

Field A (mathematical branch): Group Theory: Finitely Generated Groups **Field B** (complexity object): Resolution Lower Bounds for Tseitin Formulas

Statement:

[‘For any Tseitin formula φ on an n -vertex graph G , the minimal number of generators $m(G)$ required to generate φ in a finitely generated group is such that the Resolution length $L(\varphi)$ of φ satisfies $L(\varphi) \geq 2^{\Omega(m(G))}$.’, ‘Equivalently, for any integer $k \geq 1$, there exists an explicit graph G such that for every Tseitin formula φ on G with minimal number of generators $\leq k$, the Resolution length $L(\varphi)$ is at most $2^{O(k)}$.’, ‘Finally, if a graph G does not have a finite group presentation with fewer than $m(G)$ generators, then any Tseitin formula φ on G must have a Resolution length of at least $2^{\Omega(m(G))}$.’]

Rationale (proposer’s reasoning):

[‘This conjecture links the algebraic structure of finitely generated groups to the complexity of reasoning about Tseitin formulas. It suggests that the hardness of SAT instances in terms of Resolution complexity can be characterized by the difficulty of generating the formula within a group.’, ‘The invariant $m(G)$ captures the combinatorial properties of G , which are relevant to the structure of Tseitin formulas. If the conjecture is true, it would provide a novel approach to proving lower bounds on Resolution complexity for Tseitin formulas.’, ‘This connection between group theory and computational complexity has not been extensively explored in previous work, making this invariant a potentially fertile ground for new insights.’]

Taxonomy category: AVG_T0_WORST_CASE (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 113f49c5b3f99838

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

A graph G with n vertices is tested to see if its Tseitin formula ϕ has a Resolution length $L(\phi)$ satisfying $L(\phi) \geq 2^{\Omega(m(G))}$, where $m(G)$ is the minimal number of generators and $\Omega(m(G))$ is an asymptotic lower bound on $m(G)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal number of generators AND Finitely Generated Groups AND Resolution lower bounds-Tseitin formulas AND Group Theory AND Resolution length-Invariant for Tseitin resolution lower bounds AND graph generators

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 20 # Number of vertices in the graph
    m = 3 # Number of edges per vertex (to ensure connectivity)

    # Generate a random connected graph with n vertices and m edges per vertex
    graph = {i: set() for i in range(n)}
    for _ in range(m * n):
        u, v = random.sample(range(n), 2)
        if u != v and v not in graph[u]:
            graph[u].add(v)
            graph[v].add(u)

    # Compute the minimal number of generators m(G) using a simple heuristic
    # This is a placeholder for an actual algorithm to compute m(G)
    m_G = len(graph) # Simplified heuristic

    # Construct Tseitin formula  $\phi$  on the graph G
    clauses = []
    literals = {i: [] for i in range(n)}
    for u in range(n):
        literals[u].append(f'x_{u}')
```

```

    for v in graph[u]:
        literals[v].append(f'x_{v}')

# Add clauses to ensure each literal is true at least once
for u in range(n):
    clauses.append([f'x_{u}'])

# Add clauses to ensure no two adjacent vertices have the same literal
for u in range(n):
    for v in graph[u]:
        clauses.append([-f'x_{u}', f'x_{v}'])
        clauses.append([f'x_{u}', -f'x_{v}'])

# Compute the Resolution length  $L(\phi)$ 
def resolution_length(clauses):
    A = []
    for clause in clauses:
        row = [0] * (2 * n + 1)
        for literal in clause:
            if literal[0] == 'x':
                index = int(literal[2:]) + n
            else:
                index = int(literal[2:])
            row[index] = 1
        A.append(row)

    def rank_of_matrix(matrix):
        rows, cols = len(matrix), len(matrix[0])
        rank = 0
        for i in range(cols):
            if matrix[i][i] == 0:
                found_nonzero = False
                for j in range(i + 1, rows):
                    if matrix[j][i] != 0:
                        matrix[i], matrix[j] = matrix[j], matrix[i]
                        found_nonzero = True
                        break
                if not found_nonzero:
                    continue

            pivot = Fraction(matrix[i][i])
            for j in range(i, cols):
                matrix[i][j] /= pivot

            for j in range(rows):
                if j != i and matrix[j][i] != 0:
                    factor = -Fraction(matrix[j][i], matrix[i][i])
                    for k in range(i, cols):
                        matrix[j][k] += factor * matrix[i][k]
        return sum(1 for row in matrix if any(row))

    rank = rank_of_matrix(A)
    return rank

L_phi = resolution_length(clauses)

```

```

# Check if the Resolution length satisfies the inequality
conjecture_holds = L_phi >= 2 ** (m_G * 0.5) # Simplified heuristic for demonstration

return {
    "metric_name": "Resolution Length",
    "metric_value": L_phi,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"Graph with m(G)={m_G}, L(φ)={L_phi}"
}

if __name__ == "__main__":
    import sys
    seeds = list(map(int, sys.argv[1:])) or [2**i for i in range(5, 8)] # Default to first 3 primes

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_dev = (sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))**0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        counterexample = next(r["counterexample"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a838ed1d.py", line 113, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a838ed1d.py", line 50, in run_trial
    clauses.append([-f'x_{u}', f'x_{v}'])
                  ~~~~~^
TypeError: bad operand type for unary -: 'str'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying the conjecture’s claim. | next: Investigate and fix the error in the test code to allow for proper verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17076
2	preregistration	ollama_remote	glm4:latest	0	0	10210
3	novelty	ollama_remote	glm4:latest	0	0	8546
4	novelty	ollama_remote	glm4:latest	0	0	9054
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17541
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9822
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12134
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13020
9	judge	ollama_remote	glm4:latest	0	0	11419

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 108823 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/2e69f7d2fd43.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/2e69f7d2fd43.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/2e69f7d2fd43.tar.gz (if generated)